Advanced Database Systems Portfolio Documentation

MIT 8103 | 2026/2027 First Semester Continuous Assessment

STUDENT	Timothy Okooboh	STUDENT ID	301915902
CASE STUDY	AsterMart Retail & Fulfilment Ltd.	DATABASES	PostgreSQL 16 + MongoDB 7

Repository: github.com/timothyokooboh/advanced-database-portfolio

Organisation and technical approach

AsterMart is a simulated Nigerian multi-branch retailer with online ordering, branch pickup, delivery, electronic payment, and stock management. PostgreSQL is the transactional system of record. MongoDB supplies read-oriented product catalogue and event-timeline models. Docker Compose makes the work reproducible locally.

1. Database design and modelling

The design defines ten normalized relations for branches, categories, products, inventory, customers, addresses, orders, order items, payments, and fulfilment. The ER model records cardinalities and keys; PostgreSQL constraints enforce natural identifiers, ownership, controlled statuses, positive values, and delivery-address rules. The schema is in 3NF, while order-line prices are intentional historical snapshots.

2. Query processing and optimisation

A deterministic dataset contains 20,000 customers, 60,000 orders, 180,000 order items, and 60,000 payments. Five queries were executed with EXPLAIN (ANALYZE, BUFFERS) before and after workload-specific indexes. The largest measured improvement changed a 60,000-row branch-workload scan from 4.017 ms to a 0.034 ms index-only scan. Plans, results, buffers, index sizes, and trade-offs are retained in the repository.

EVIDENCE	BASELINE	OPTIMISED	OBSERVATION
Branch workload	4.017 ms	0.034 ms	Index-only range scan
Product sales	8.029 ms	1.118 ms	Avoided 180,000-line scan
Payment window	3.205 ms	0.198 ms	Partial successful-payment index

Practical implementation and advanced models

3. Transactions and concurrency

The place_order function locks inventory rows in deterministic product order, validates stock, inserts the header and lines, and deducts inventory atomically. In a two-session test, both sessions requested four of five units. Session A committed; Session B waited about 1.8 seconds, observed the new balance of one, and rolled back with an insufficient-stock error. A separate rollback test restored an inventory value from 141 to 143.

4. NoSQL and advanced data models

MongoDB stores 500 product catalogue documents with flexible category attributes and embedded branch availability, plus 5,000 append-oriented order events. Strict JSON Schema validators reject malformed documents. Indexes support SKU identity, category/price browsing, tags, branch availability, text search, and timelines. An explain run used idx_category_price, examining 32 keys and 32 documents for 32 results. PostgreSQL remains authoritative for checkout inventory.

5. Distributed and cloud database exercise

Two PostgreSQL containers implement logical replication from an operational publisher to a reporting subscriber. Initial data and a later branch-summary change replicated successfully; publisher and subscriber values matched and pg_stat_subscription showed active receive/apply positions. A read-only reporting account could query the subscriber but its insert was rejected.

ORDER / INVENTORY WRITES	LOGICAL REPLICATION	REPORTING READS
PostgreSQL publisher port 5441	Publication + WAL changes asynchronous	PostgreSQL subscriber port 5442

The subscriber improves reporting isolation and read availability but is eventually consistent and is not automatic high availability. Production deployment should add multi-zone failover, encrypted private networking, secret management, monitored lag and retained WAL, backups with restore drills, and coordinated schema migrations. Branch-oriented sharding is proposed only after measured scale justifies its operational cost.

Verification highlights

- PostgreSQL and MongoDB run locally through Docker Compose with deterministic initialization.
- Automated checks cover row counts, totals, constraints, indexes, rollback postconditions, document diversity, and replication.
- Evidence files preserve query plans/results, concurrency output, validation output, and replication status.
- The same AsterMart case study is used consistently across all five activities.

Repository evidence map

PORTFOLIO	PRIMARY EVIDENCE
1 - Design	ERD, DDL, keys, constraints, normalisation
2 - Optimisation	SQL results, before/after plans, index analysis
3 - Transactions	Commit/rollback output, competing-session lock test
4 - NoSQL	MongoDB model, queries, explain and validation
5 - Distributed	Publisher/subscriber output and monitoring state

Reflection, declaration and references

Individual technical reflection

This portfolio showed me that advanced database design is not only about creating tables. Constraints and normalization establish correctness, but workload evidence determines useful indexes, and transaction boundaries determine whether a correct schema remains correct under concurrency. The first seed run failed because a temporary discounted order violated the total constraint; executing the scripts exposed this early. The scarce-stock test made row locks and isolation concrete because the second session waited and rechecked committed stock instead of overselling.

I also learned to choose models by access pattern. MongoDB simplified variable product attributes and event payloads, but duplicating availability creates a consistency boundary, so PostgreSQL must remain authoritative during checkout. Logical replication improved reporting isolation and read availability, yet it did not remove the need for backups, schema coordination, lag monitoring, or planned failover. My main lesson is that design claims should be supported by repeatable scripts, measured plans, negative tests, and documented limitations.

AI use declaration

OpenAI Codex assisted with milestone planning, drafts of code and documentation, testing ideas, and formatting. All scripts were executed locally; outputs and database states were inspected, and generated material was corrected when tests exposed problems. Official PostgreSQL and MongoDB documentation was used for technical claims. I remain responsible for reviewing, understanding, explaining, and submitting the work. The repository contains the full declaration.

References

1. PostgreSQL Global Development Group. PostgreSQL 16 Documentation. https://www.postgresql.org/docs/16/
2. PostgreSQL Global Development Group. Logical Replication. https://www.postgresql.org/docs/16/logical-replication.html
3. MongoDB, Inc. Data Modeling. https://www.mongodb.com/docs/manual/data-modeling/
4. MongoDB, Inc. Schema Validation. https://www.mongodb.com/docs/manual/core/schema-validation/

Submission note: The repository is private and must remain accessible to the assessor. The student should review this reflection and declaration in their own words before uploading the PDF through the LMS.